

memory address. Therefore, the reference count of the object is now 2. If we were to assign another variable:

```
oSquare3 = oSquare2 # or oSquare1
```

the reference count would be incremented to 3 (because all three variables would refer to the same object), and so on.

The reference count of an object is important because when it reaches zero, Python marks the relevant memory as no longer in use by the program. This is known as being *marked as garbage*. Python has a garbage collector that runs to reclaim any blocks of memory that have been marked as garbage; I'll discuss that later in this chapter.

The Python Standard Library contains the `getrefcount()` function, which returns the number of variables that refer to an object. Here, we use it to see the reference count after the first instantiation of a `Square` object from the `Square` class:

```
oSquare1 = Square(5, 'red')
print('Reference count is', sys.getrefcount(oSquare1))
```

This prints a count of 2. This may be surprising—you likely expected that the count would be 1. However, as the documentation of this function explains, “The count returned is generally one higher than you might expect, because it includes the (temporary) reference as an argument to `getrefcount()`.”

Incrementing the Reference Count

There are a few ways that an object's reference count is incremented:

1. When an additional variable is assigned to refer to the same object:

```
oSquare2 = oSquare1
```

2. When an object is passed into a function and therefore a local parameter variable is set to refer to the object: